## Clone now!

# github.com/mniehoff/duckdb-beyond-the-notebook

run `uv sync` in `backend`, `server` and `spark`

codecentric

# duckdb beyond the notebook



# The plan

- duckdb by examples
- as cli helper
- as ai tool
- in a lambda (demo only)

- in the backend
- in the browser
- as a server
- what to do with it?

DuckDB is a
fast
in-process
analytical
open source
portable
database system

from https://duckdb.org/

# We start hands-on

```
duckdb --ui
```

# Analyse local

```
SELECT *
FROM   read_json("employees.json");


SELECT department,
       avg(salary)
FROM   read_json("employees.json")
GROUP  BY all;
```

# Analyse remote

```
SELECT *
FROM   read_parquet("https://…/oil-and-gas.parquet");


SELECT "reporting year"          AS year,
       sum("gas produced, mcf") AS gas
FROM   read_parquet("https://…/oil-and-gas.parquet" )
GROUP  BY "reporting year";


SELECT *
FROM   read_json("https://api.spacexdata.com/v3/launches")
```

# Analyse blob

```
SET s3_region='eu-central-1';
SET s3_access_key_id='AKIA2RYWYEMUJVK3C4X4';
SET s3_secret_access_key='';

SELECT * EXCLUDE payload
FROM read_json("s3://test-data-matthias/github/*.json.gz")

SELECT actor.login, actor.url FROM
read_json("s3://test-data-matthias/github/*.json.gz")
```

# Portable

- no external dependencies
- a lot of clients
- every platform - linux, windows, mac



```
pip install duckdb
```




```
org.duckdb:duckdb_jdbc
```



```
install.packages("duckdb")
```




```
npm install duckdb
```

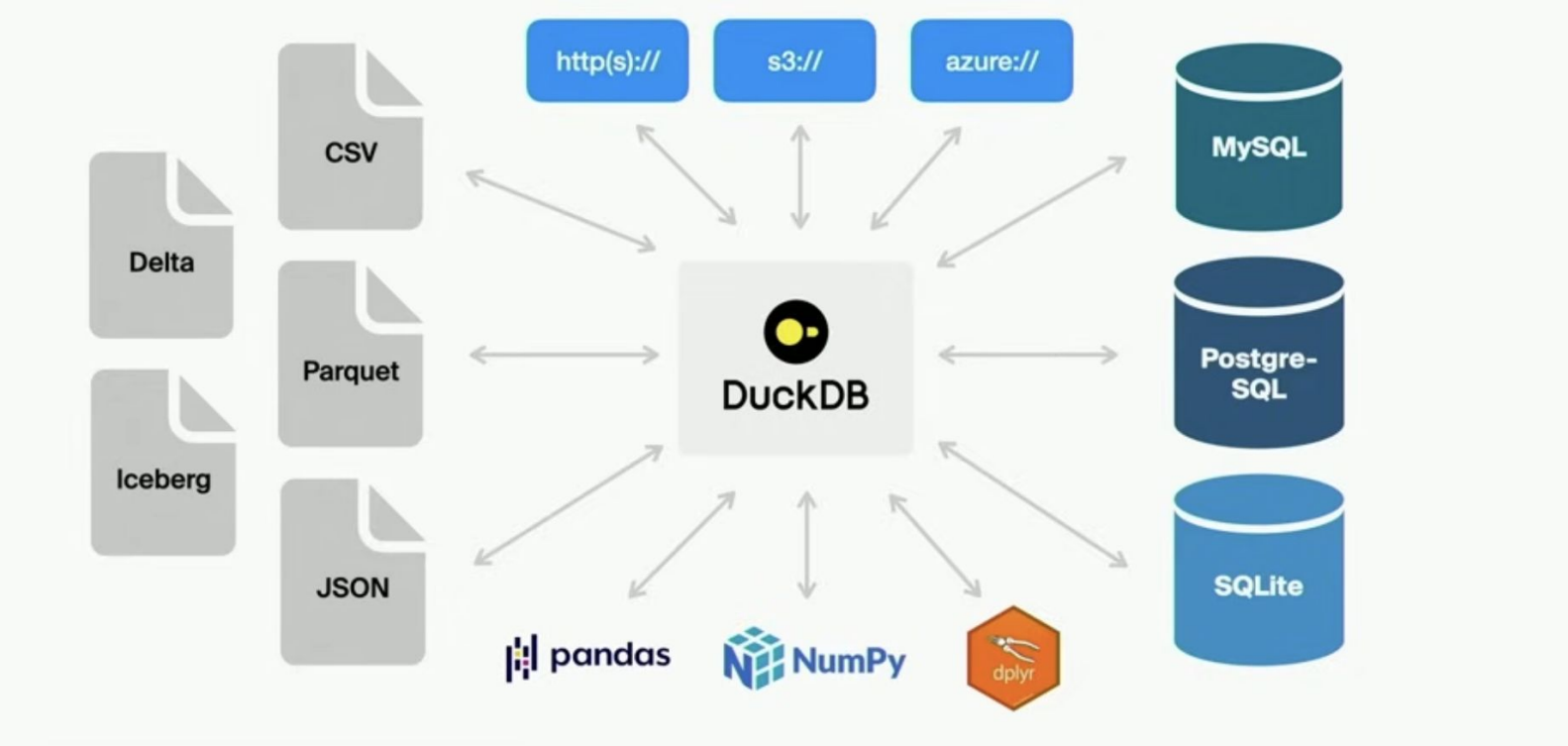


```
brew install duckdb
```



```
cargo add duckdb
```

# Flexible




DuckDB: Crunching Data Anywhere, From Laptops to Servers • Gabor Szarnyas • GOTO 2024
https://youtu.be/9Rdwh0rNaf0

# Work Time

First: Logfiles

# duckdb for AI Data Access

hands-on

https://github.com/motherduckdb/mcp-server-motherduck

# duckdb ETL in a Lamba

demo

# duckdb to replace spark workloads

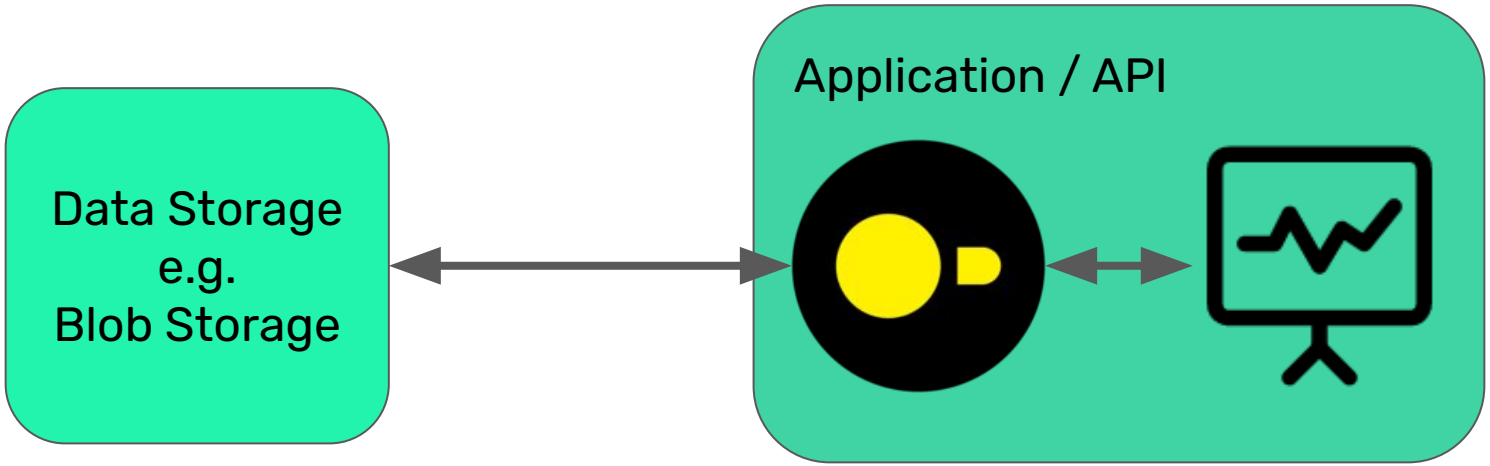
```
from sqlframe import activate

# Activate SQLFrame to run directly on DuckDB
activate(engine="duckdb")

from pyspark.sql import SparkSession
session = SparkSession.builder.getOrCreate()
```

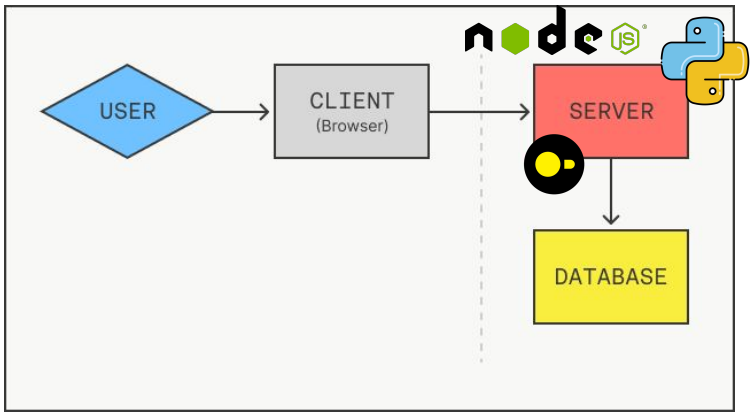
# Data Applications / API

The query engine for a fast user experience



# The “usual” way:

## traditional 3-tier



# duckdb in the backend

hands-on

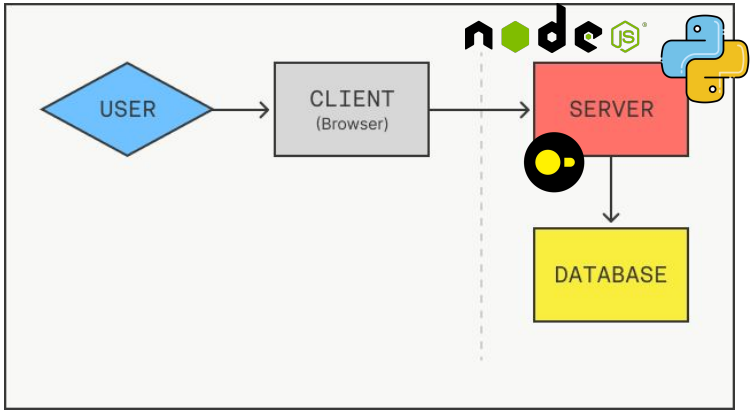
# duckdb in the backend

- Query Engine in the backend
- Storage could be (cheap) object storage
- Backend could be run
  - once
  - or even for every client
  - scaled to zero
- Still the classic three tier architecture
  - Storage
  - Backend
  - Frontend

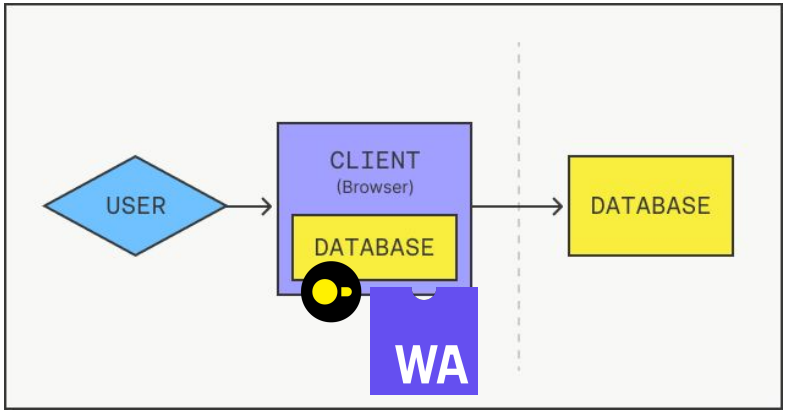
- But query engine has been decoupled from the storage

# Going Beyond 3 Tier Data Architectures" ?

## traditional 3-tier




## client side computing



# Data App in the browser with WASM

hands-on

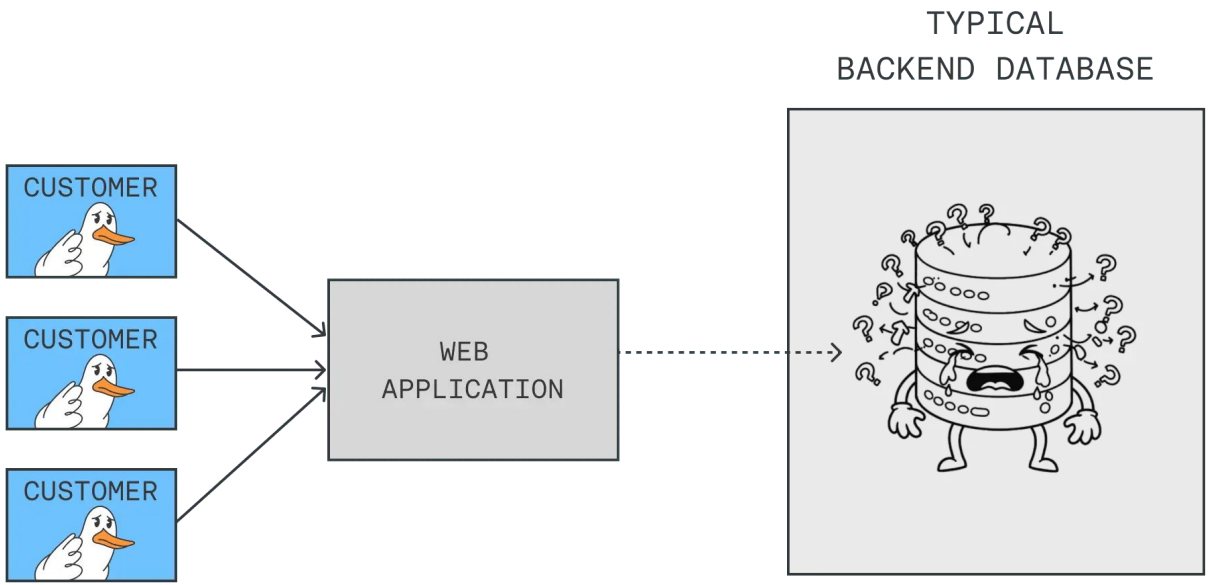
# WASM Data Application

- Plenty of examples and applications already

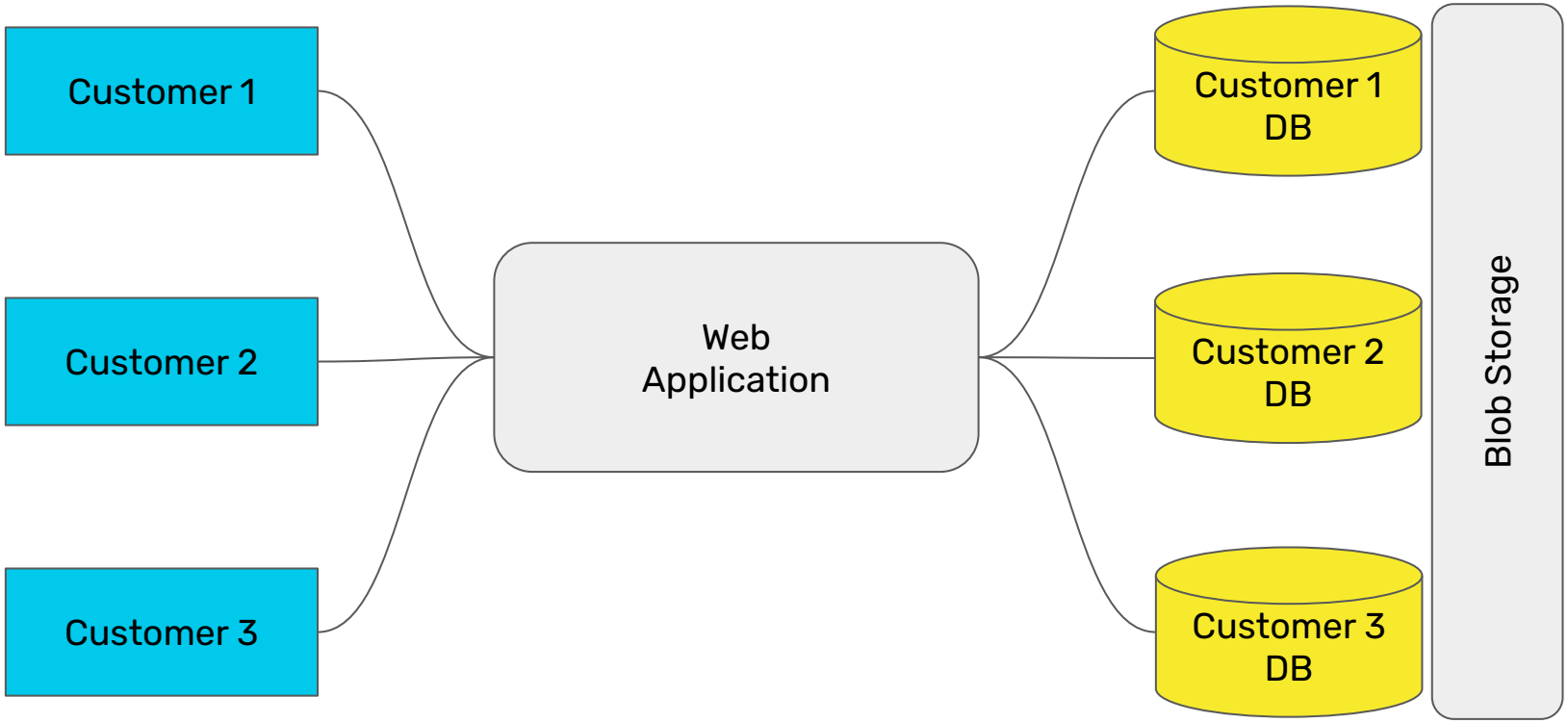
- TypeScript + WebAssembly library
- WebShell

- Caveats:
  - Single Threaded & WASM Memory Limits
  - Beware of CORS
  - No local file access
  - Some extensions not available

# Typical Analytical Applications



# Scaling a Analytical Applications



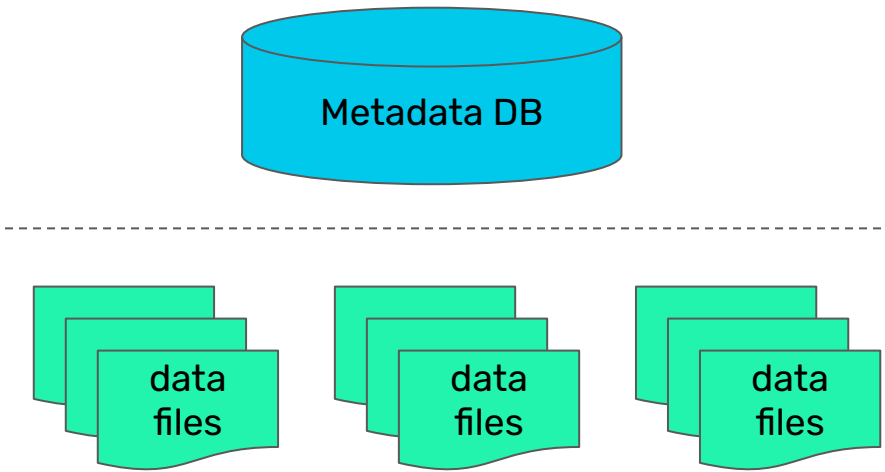

https://motherduck.com/blog/scaling-duckdb-with-ducklings/

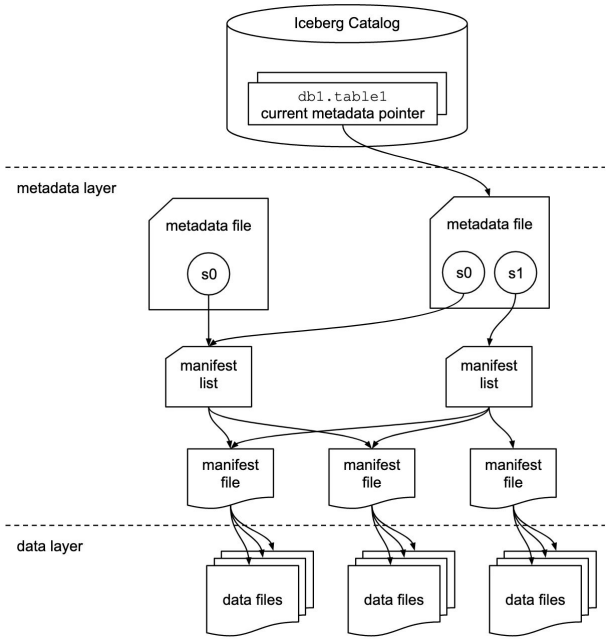
# It’s not only a library (anymore)

# ducklake - “Lakehouse” format

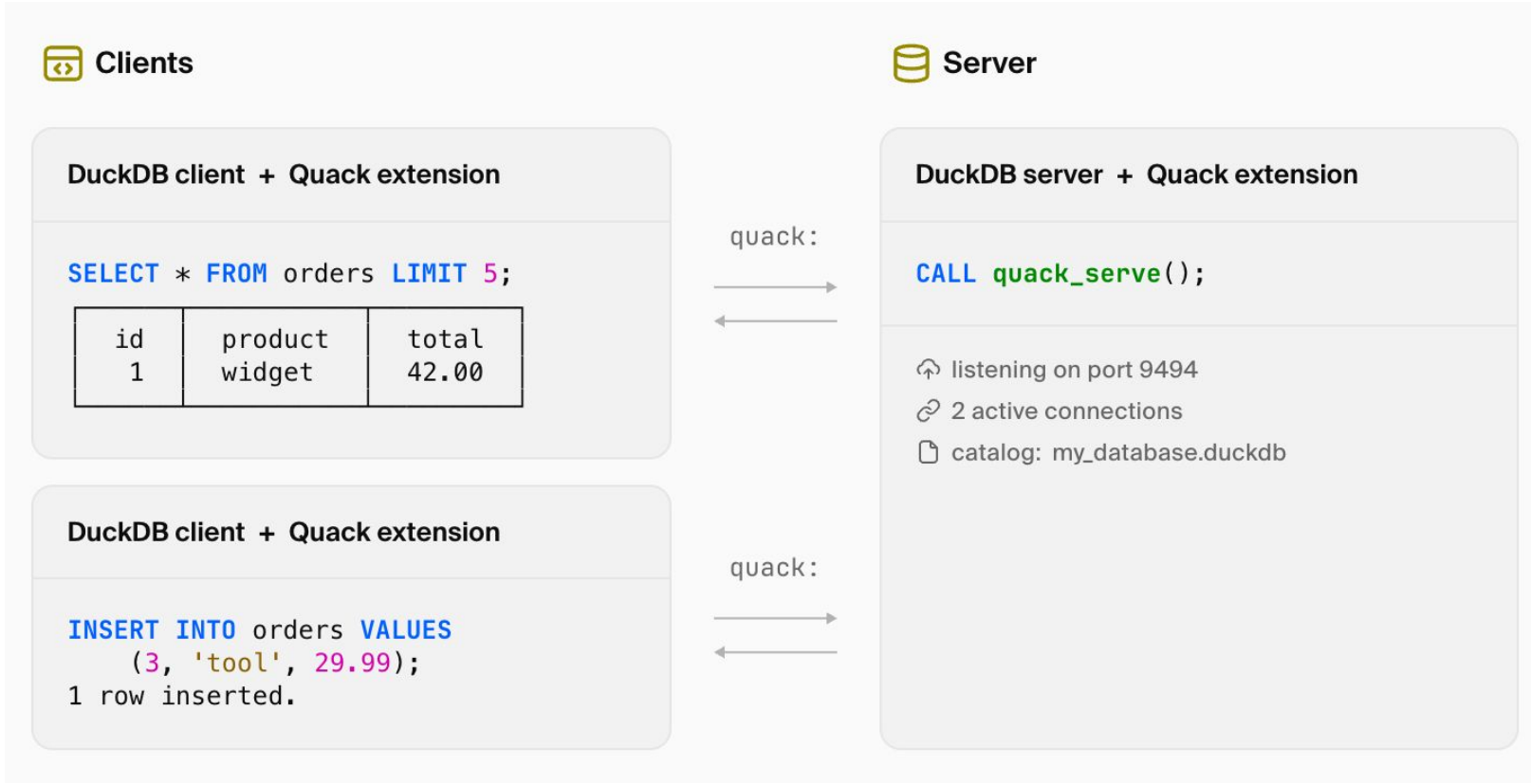
DuckLake

- duckdb’s take on a modern table format
- Snapshots, transactions, time travel, schema evolution …



# Quack Protocol (beta)





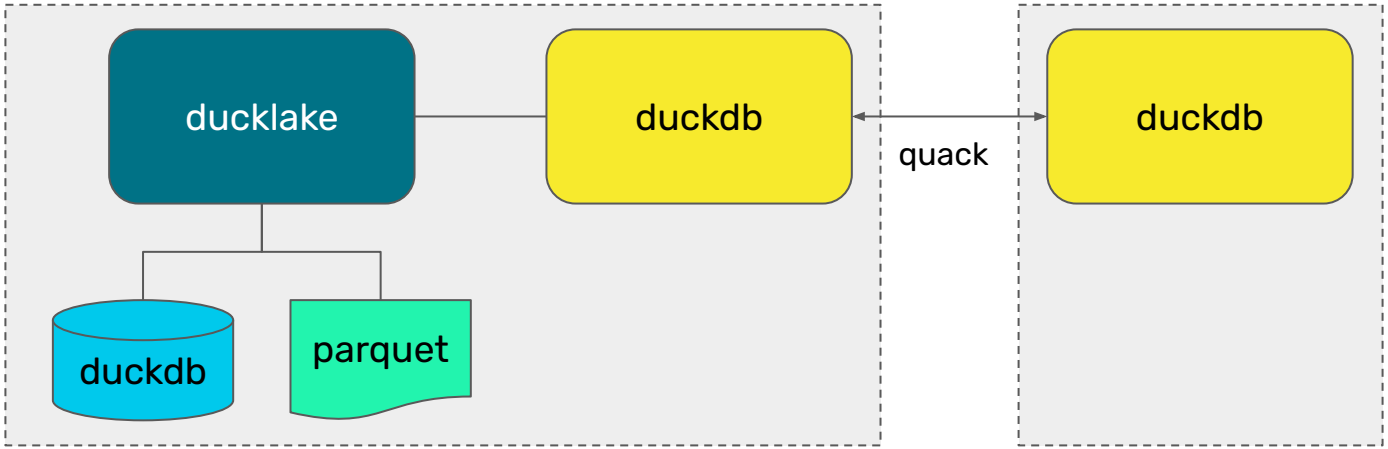

https://duckdb.org/quack/

# duckdb as a server database for analytics

hands-on

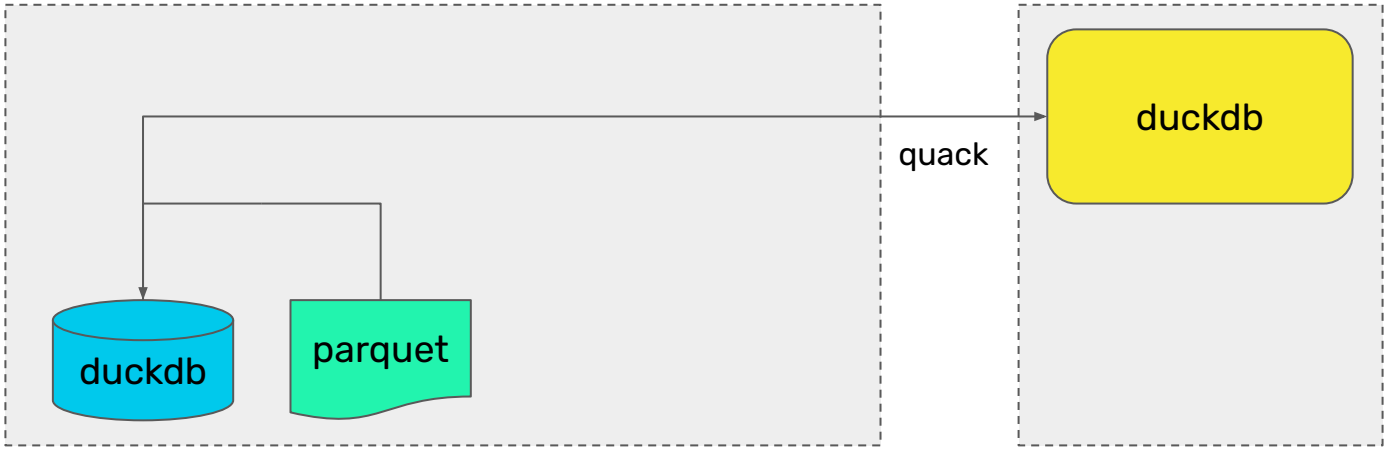
# Data Application

- Using duckdb as server database
- Quack + ducklake for multi user access & scaling



# Alternative

- connect to duckdb catalog database
- client executes the query
- before: server duckdb executed the query, returns resultset



# Why use it?

- made for analytical workloads
- duckdb power on the client
  - post process before inserting
  - aggregate on the client
  - create temp table on the client for caching
  - create a local read replica
    (replication is planed for quack)

- compared to “duckdb embedded”
  - supports multiple source of updates
  - easier to operate than “distribute files”

# duckdb 2.0 - this fall

- better quack integration (CONNECT syntax to replace remote.query)
- VARIANT improvements (Shredding)
- TRIGGERS
- SQL Additions, including nested schemas
- Async IO, e.g. for parquet
- Performance improvements
- New SQL Parser
- Stable Extension API

# What to do with duckdb?



# Local prototyping & exploration

01 - Analyse local

Run memory

```
1  SELECT *
2  FROM   read_json("/Users/matthiasniehoff/dev/git/duckdb/data/simple/employees.json");
3
```

Run memory

```
1  SELECT department,
2         avg(salary)
3  FROM   read_json("/Users/matthiasniehoff/dev/git/duckdb/data/simple/employees.json")
4  GROUP  BY all;
```

# Single Node Data Processing Engines



pandas

DuckDB

PySpark

single node
not bigger than
memory

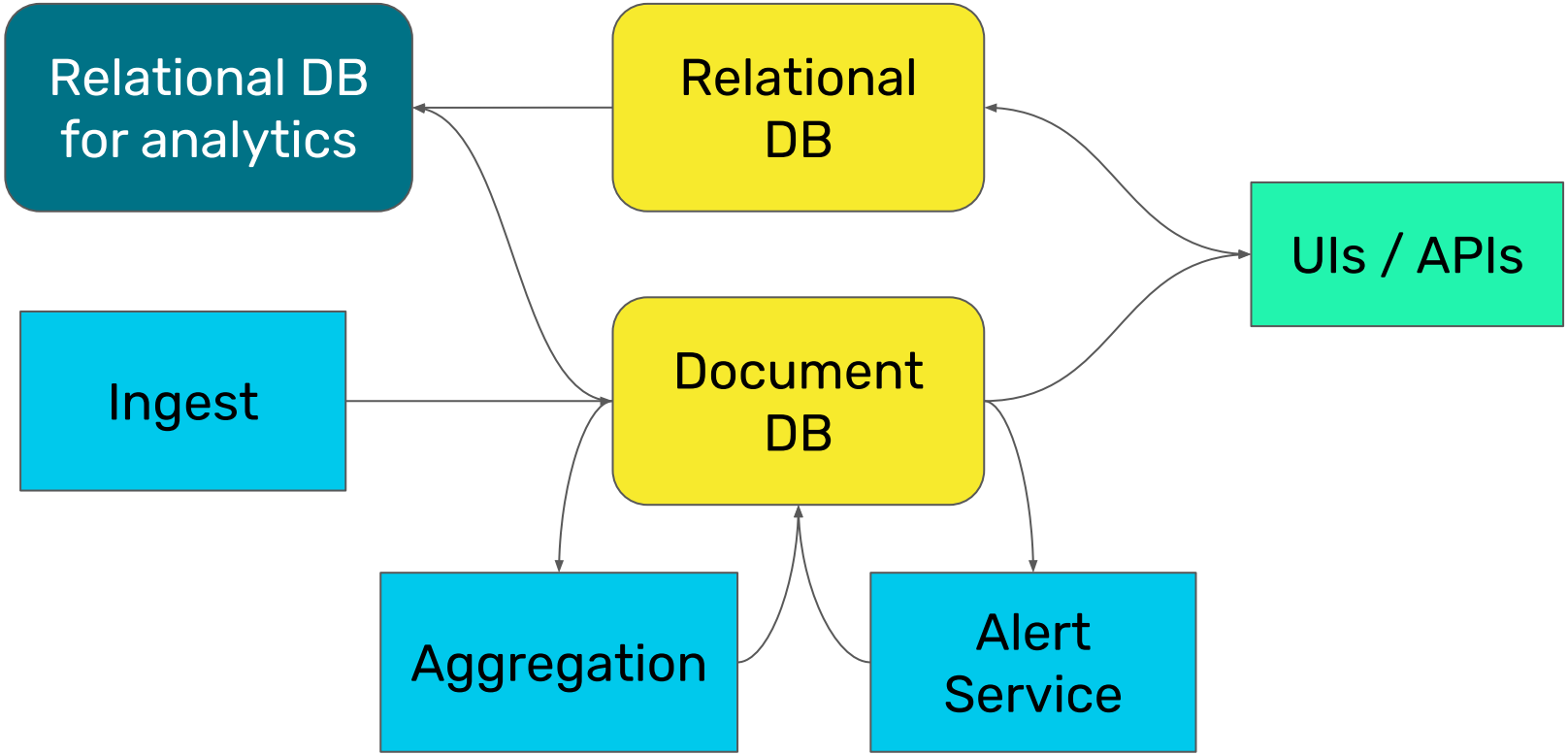
single node
bigger than
memory

multi node
linear
scaling

# Simpler data driven applications

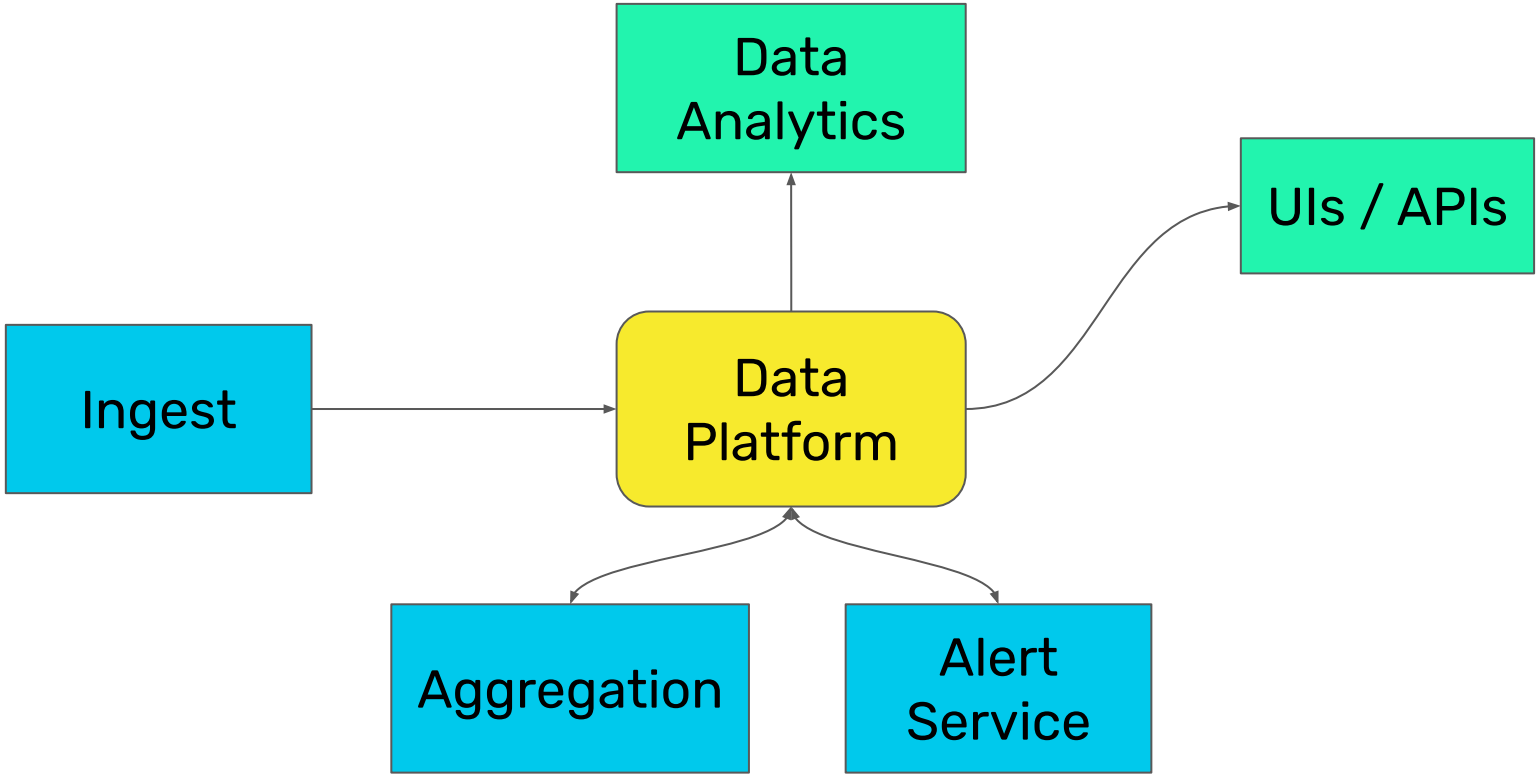


# What we had

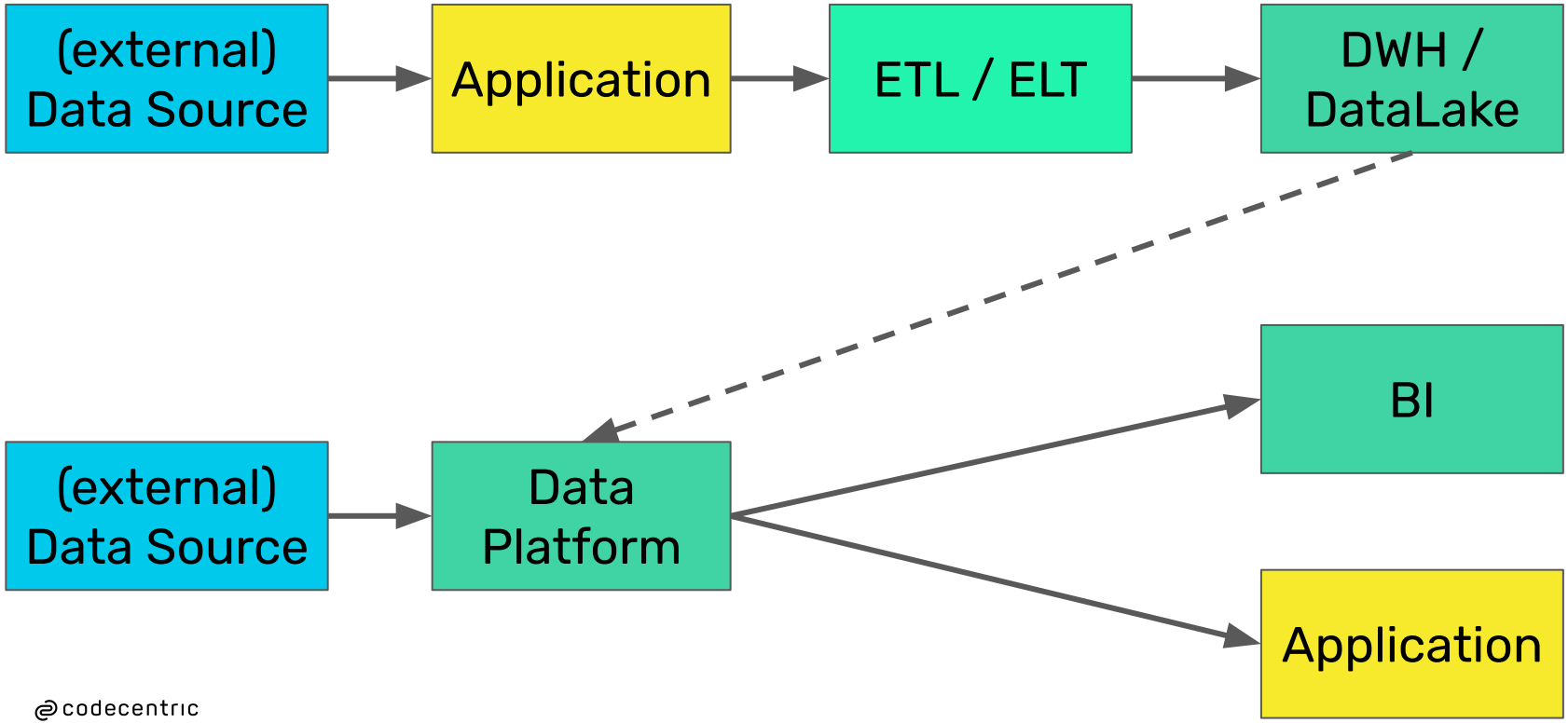

Relational DB for analytics
Relational DB
UIs / APIs
Ingest
Document DB
Aggregation
Alert Service

# The web application is actually a data platform with an UI

# What we have now



# Shift left in data flows




Takeaways

# Conclusion

- duckdb is not only for analysts
- used to build data driven / analytical applications
- powers data pipelines in large companies
- versatile, flexible and friendly to use
- might go for “basic” transactional workloads eventually



## Repository & Slides



# mniehoff/ duckdb-beyond-the-notebook

**We support you with your data projects**

**Feel free to contact us!**

**codecentric AG**
**Hochstraße 11**
**42697 Solingen**

**Matthias Niehoff**
Principal Data Architect
**matthias.niehoff@codecentric.de**
**www.codecentric.de**

codecentric

